

Laboratorio 1 - Introducción a MATLAB y Live Editor

Integrantes: Mateo Fernández (202320768), Gabriela Villalba (202411276) y Gabriel Sánchez (202420981)

```
clc;
clearvars;
close all;
format long g

carpetaActual = pwd;
versionMATLAB = version;
variablesIniciales = whos;
```

Punto 1

1.1

```
d = [2 0 2 4 2 0 9 8 1]
```

```
d = 1×9
     2     0     2     4     2     0     9     8     1
```

```
p = d(1,7)+2
```

```
p =
    11
```

```
q = d(1,8)+3
```

```
q =
    11
```

```
r = d(1,9)+4
```

```
r =
     5
```

```
theta = 10*d(1,4)+d(1,5)
```

```
theta =
    42
```

1.2

```
E1_grados = (sind(theta)+(cosd(p+q))^2+r^(-1))/(log10(p^2+q^2+10)+(log(r+20))/(sqrt(p^2+q^2+r^2)))
```

```
E1_grados =
    0.665334616880019
```

```
E2_grados = ((p+q)^2/(tand(theta+5)))*log(r+1)+power(((p+r)*exp(-q/10))/log2(p*q+r+2),1/3)
```

```
E2_grados =
    809.600804013371
```

```
E1_radianes = (sin(theta)+(cosd(p+q))^2+r^(-1))/(log10(p^2+q^2+10)+(log(r+20))/(sqrt(p^2+q^2+r^2)))
```

```
E1_radianes =
```

0.0550911188098749

```
E2_radianes = ((p+q)^2/(tan(theta+5)))*log(r+1)+power(((p+r)*exp(-q/10))/log2(p*q+r+2),1/3)
```

```
E2_radianes =  
-6963.0999176656
```

Efecto de interpretar θ con una unidad angular diferente:

El hecho de utilizar *sind* vs *sin* puede tener implicaciones significativas dentro del calculo. La función seno no es lineal y un cambio ligero en el calculo de este puede tener implicaciones significativas, por ejemplo, el angulo de 45° es $\frac{1}{8}$ de circulo por lo que el seno de 45° es $\frac{\sqrt{2}}{2}$, sin embargo 45 radianes es el equivalente de aproximadamente 7.16 vueltas al circulo unitario, lo que es aproximadamente 57° . Por lo que comprender bajo que escala se mide el angulo es importante para prevenir inconsistencias numericas.

1.3

```
T = table(p', q', r', theta', E1_grados', E2_grados', E1_radianes', E2_radianes', ...  
    'VariableNames', {'p', 'q', 'r', 'theta', 'E1_grados', 'E2_grados', 'E1_radianes', 'E2_radianes'})
```

```
T = 1x8 table
```

	p	q	r	theta	E1_grados	E2_grados	E1_radianes
1	11	11	5	42	0.665334616880019	809.600804013371	0.0550911188098749

```
format short  
E1_grados
```

```
E1_grados =  
0.6653
```

El formato short se encarga de mostrar numeros con maximo 4 cifras decimales, es estético y solo afecta lo que muestra la consola, internamente los numeros se manejan como doubles y son los tipos de datos numericos estandar del programa. Estos proveen alrededor de 15-17 cifras decimales de precisión. Matlab al ser un lenguaje de programación con tipado dinamico (asigna el tipo de variable de forma dinamica) no se tiene que especificar el tipo de dato a ingresar, sin embargo si se requiere o se prefiere algún tipo de dato en especifico siempre se pueden convertir los datos double a single o a enteros o algun dato de preferencia.

```
format long;  
class(E1_grados)
```

```
ans =  
'double'
```

```
E1_grados
```

```
E1_grados =  
0.665334616880019
```

Por el contrario el formato long se encarga de mostrar resultados con hasta 15 cifras decimales

```
fprintf("%4.6f", E1_grados)
```

```
0.665335
```

```
fprintf("%2.8f", E1_grados)
```

```
0.66533462
```

En el formato fprintf permite escribir datos a un archivo de texto o imprimir directamente a la consola si se omite la ruta al archivo de texto a editar. De manera similar al lenguaje de programación C, se utilizan ciertos operadores dentro de un string, en este caso el % indica "reemplazar por cierta variable" e inmediatamente después se especifica el tipo de variable y/o el formato con el que se muestra. Como se está manejando un número (tipo float) se especifican la cantidad de números enteros a mostrar y la cantidad de decimales fijos a mostrar. Si se quiere mostrar un entero se utilizaría %d, o si se quisiera en notación científica se utilizaría %e, o una cadena de caracteres sería %s.

```
log(0)
```

```
ans =  
-Inf
```

```
sqrt(-1)
```

```
ans =  
0.000000000000000 + 1.000000000000000i
```

A pesar de que ciertas funciones tienen restricciones en su dominio como los logaritmos o las raíces, Matlab puede llegar a manejar de forma completa algunas de estas funciones. Por ejemplo calcular el logaritmo natural de 0 resulta en -Inf, pero no en error y la raíz cuadrada de -1 resulta en 1i y no en error, por lo que a pesar de que pueden dar valores poco útiles, no afecta en la compilación del programa.

Punto 2

2.1

Función auxiliar reporte de información de los arreglos

```
function info_arreglo(nombre_arreglo, arreglo)  
    tamaño = size(arreglo);  
    num_filas = tamaño(1);  
    num_columnas = tamaño(2);  
    cantidad_elementos = numel(arreglo);  
    tipo_dato = class(arreglo);  
    valor_min = min(arreglo(:));  
    valor_max = max(arreglo(:));  
    memoria = whos('arreglo');  
    memoria_ocupada = memoria.bytes;  
    fprintf('%s\n', nombre_arreglo);  
    fprintf('Tamaño: %d filas x %d columnas\n', num_filas, num_columnas);  
    fprintf('# de elementos: %d\n', cantidad_elementos);  
    fprintf('Clase: %s\n', tipo_dato);  
    fprintf('Mínimo: %g | Máximo: %g\n', valor_min, valor_max);  
    fprintf('Memoria ocupada: %d bytes\n\n', memoria_ocupada);  
end
```

```
A = [p q r; q+r p+1 q-1; r-p q+2 p+r]
```

```
A = 3x3
```

11	11	5
16	12	10
-6	13	16

```
B = [r+1 p-1 q; p+q r 1; q-r p+2 r+2]
```

```
B = 3x3
      6      10      11
     22      5       1
      6      13       7
```

```
vector_fila = [p q r p+r];
vector_columna = [p; q; r; p+r];
matriz_diagonal = diag([p q r]);
matriz_toep = toeplitz([p q r]);
info_arreglo('Vector fila', vector_fila);
```

```
Vector fila
Tamaño: 1 filas x 4 columnas
# de elementos: 4
Clase: double
Mínimo: 5 | Máximo: 16
Memoria ocupada: 32 bytes
```

```
info_arreglo('Vector columna', vector_columna);
```

```
Vector columna
Tamaño: 4 filas x 1 columnas
# de elementos: 4
Clase: double
Mínimo: 5 | Máximo: 16
Memoria ocupada: 32 bytes
```

```
info_arreglo('Matriz diagonal', matriz_diagonal);
```

```
Matriz diagonal
Tamaño: 3 filas x 3 columnas
# de elementos: 9
Clase: double
Mínimo: 0 | Máximo: 11
Memoria ocupada: 72 bytes
```

```
info_arreglo('Matriz de Toeplitz', matriz_toep);
```

```
Matriz de Toeplitz
Tamaño: 3 filas x 3 columnas
# de elementos: 9
Clase: double
Mínimo: 5 | Máximo: 11
Memoria ocupada: 72 bytes
```

2.2

```
suma = A + B
```

```
suma = 3x3
      17      21      16
      38      17      11
       0      26      23
```

El Significado Matematico de $A + B$ es simplemente la suma individual de cada uno de los indices correspondientes, es decir el indice a11 se suma con b11 y el indice a12 se suma con b12 y asi sucesivamente.

```
producto_AB = A*B
```

```
producto_AB = 3x3
    338    230    167
    420    350    258
    346    213     59
```

Al utilizarse la multiplicación entre matrices, se define que el resultado entre la multiplicación de matrices tendra una dimesión $a*c$ cuanto las matrices de entrada tienen dimensión $(a,b*b,c)$, note que si la dimension del "ancho" de la primera matriz no coincide con la dimension del "alto" de la segunda matriz la multiplicación matricial NO está definida. La multiplicación matricial NO es conmutativa debido al algoritmo que se utiliza para calcularla (fila*columna) en donde al cambio en el orden de los indices afecta el resultado de la multiplicación fila*columna. Para verificar, se calcula el resultado c11 (el primer indice de ans, que en este caso es 338).

```
A(1,:)*B(:,1)
```

```
ans =
    338
```

Se accede a la primera fila de la matriz A utilizando slicing, es decir se accede a la primera fila y se guardan TODOS los valores de esa columna con el operador. De manera similar, se accede la primera columna de la matriz B utilizando solamente la primera columna, pero se almacenan TODAS las filas. Como el resultado de ambas operaciones da un vector, se pueden multiplicar directamente para obtener el producto punto entre estos dos vectores que da el mismo resultado que se obtuvo multiplicando las matrices completas de A y de B.

```
producto_BA = B*A
```

```
producto_BA = 3x3
    160    329    306
    316    315    176
    232    313    272
```

```
prod_Elemento = A.*B
```

```
prod_Elemento = 3x3
    66    110     55
   352     60     10
   -36    169    112
```

El producto de $A .* B$ es equivalente a la multiplicación individual entre los indices de las matrices, es decir de manera similar a $A+B$, se obtiene que $a_{11}*b_{11}$ es igual al primer indice del resultado. De manera similar, el segundo indice resultante es $a_{12}*b_{12}$ y asi sucesivamente.

```
div_Elemento = A./B
```

```
div_Elemento = 3x3
    1.83333333333333    1.10000000000000    0.454545454545455
    0.727272727272727    2.40000000000000   10.000000000000000
   -1.00000000000000    1.00000000000000    2.285714285714286
```

```
verificacion_AB_21= A(2,1)*B(1,1) + A(2,2)*B(2,1) + A(2,3)*B(3,1);
fprintf('Verificación manual AB(2,1) = %g | MATLAB AB(2,1) = %g\n', verificacion_AB_21,
producto_AB(2,1));
```

```
Verificación manual AB(2,1) = 420 | MATLAB AB(2,1) = 420
```

```
fprintf('AB(2,1) = %g (producto matricial) | A.*B(2,1) = %g (producto elemento a elemento)\n', producto_AB(2,1), prod_Elemento(2,1));
```

AB(2,1) = 420 (producto matricial) | A.*B(2,1) = 352 (producto elemento a elemento)

2.3

```
C = [4 2 7; 3 5 9];
try
    incompatible = A*C;
catch Mensaje_error
    fprintf('Mensaje de MATLAB:\n%s\n', Mensaje_error.message);
    fprintf('Dimensiones A: %dx%d | Dimensiones C: %dx%d\n', size(A,1), size(A,2), size(C,1), size(C,2));
end
```

Mensaje de MATLAB:
Incorrect dimensions for matrix multiplication. Check that the number of columns in the first matrix matches the number of rows in the second matrix.
Dimensiones A: 3x3 | Dimensiones C: 2x3

```
correccion_1 = C*A
```

```
correccion_1 = 2x3
    34    159    152
    59    210    209
```

```
disp('C*A ='); disp(correccion_1)
```

```
C*A =
    34    159    152
    59    210    209
```

```
fprintf('Corrección 1 C*A: | Dimensiones = %dx%d\n', size(correccion_1,1), size(correccion_1,2));
```

Corrección 1 C*A: | Dimensiones = 2x3

```
C_tranpuesta = C';
fprintf('Antes de la corrección 2, dimensiones de C_tranpuesta: %dx%d\n', size(C_tranpuesta,1), size(C_tranpuesta,2))
```

Antes de la corrección 2, dimensiones de C_tranpuesta: 3x2

```
correccion_2 = A*C_tranpuesta;
disp('A*C' ='); disp(correccion_2)
```

```
A*C' =
    101    133
    158    198
    114    191
```

```
fprintf('Corrección 2 A*C' | Dimensiones = %dx%d\n', size(correccion_2,1), size(correccion_2,2));
```

Corrección 2 A*C' | Dimensiones = 3x2

Al Intentar multiplicar la Matriz A1 de dimensiones (2x3) y la matriz B1 de dimensiones (2x3) la multiplicación falla debido a que no se siguen las reglas de dimensionamiento previamente explicadas en donde se debe cumplir (a,bx b,c) para que el resultado tenga dimension axc. Para arreglar este error simplemente se puede transponer la matriz B1 para que en número de columnas de A1 concuerde con el número de filas de B1. se utiliza el operador .' para transponer la matriz B1.

Punto 3

3.1

Análisis de resultados:

La función $f_1(x) = 13 \cos\left(\frac{x}{12}\right)$ tiene amplitud 13 (dada por $p+2$) y periodo $= 2\pi(q+1) = 24\pi \approx 75.4$, por lo que en el rango evaluado ($x = 0$ a 20) apenas se observa una fracción del primer cuarto de oscilación. Inicia en $f_1(0) = 13$ (máximo) y decae hasta $f_1(20) \approx -1.24$, cruzando el eje horizontal cerca de $x = 6\pi \approx 18.85$ esto explica por qué en la gráfica la curva recién empieza a cruzar el eje x justo al final del rango evaluado. Es importante notar que en nuestro código el argumento del coseno ($x/12$) se lee directamente en radianes; lo cual da un resultado completamente diferente a si lo evaluáramos en grados (consistente con la explicación del punto 1.2).

Por otro lado, la función $f_2(x) = e^{-\frac{3x}{22}}$ es una exponencial decreciente, con valor inicial $f_2(0) = 1$ y como podemos asociarla a la forma $A \cdot e^{-(x/\tau)}$ su constante de tiempo (el valor de x en el que la función cae exactamente al 36.8% ($1/e$) de su valor inicial) obtenemos $\tau = \frac{p+q}{3} = \frac{22}{3} \approx 7.33$. Esto nos dice que cada 7.33 unidades de x , la función pierde aproximadamente dos tercios de su valor restante.

En $x = 20$ (más de 2.7 constantes de tiempo), $f_2(20) \approx 0.065$ (6.5% del valor inicial de 1) prácticamente ha desaparecido (nunca cruza el eje x , coherente con que una exponencial siempre es positiva).

La función $f_3(x) = \frac{11x^2 - 11x - 5}{x + 11}$ tiene su único polo en $x = -11$, fuera del rango evaluado, por lo que no presenta discontinuidades ni divisiones por valores cercanos a cero dentro de $[0, 20]$. Por la forma del polinomio podemos asumir que frente un x cada vez más grande el término cuadrático domina, de hecho si se hace la división generando un crecimiento pronunciado, a diferencia de f_1 (acotada entre -13 y 13) y f_2 (que decae hacia 0), f_3 crece de forma sostenida $f_3(20) \approx 134.7$ el mayor valor de las tres funciones. Esto último es la misma razón por la cual cada función requiere su propio espacio o cuadrícula dado que comparten muy poca escala común.

```
x = 0:0.1:20
```

```
x = 1×201
      0      0.1000000000000000      0.2000000000000000      0.3000000000000000 ...
```

```
f1 = (p+2)*cos(x/(q+1))
```

```
f1 = 1×201
    13.000000000000000    12.999548613723309    12.998194486239324    12.995937711584133 ...
```

```
f2 = (r/5)*exp(-3*x/(p+q))
```

```
f2 = 1×201
    1.000000000000000    0.986456190392484    0.973095815563652    0.959916391107787 ...
```

```
f3 = (p*x.^2 - q*x - r)./(x+q)
```

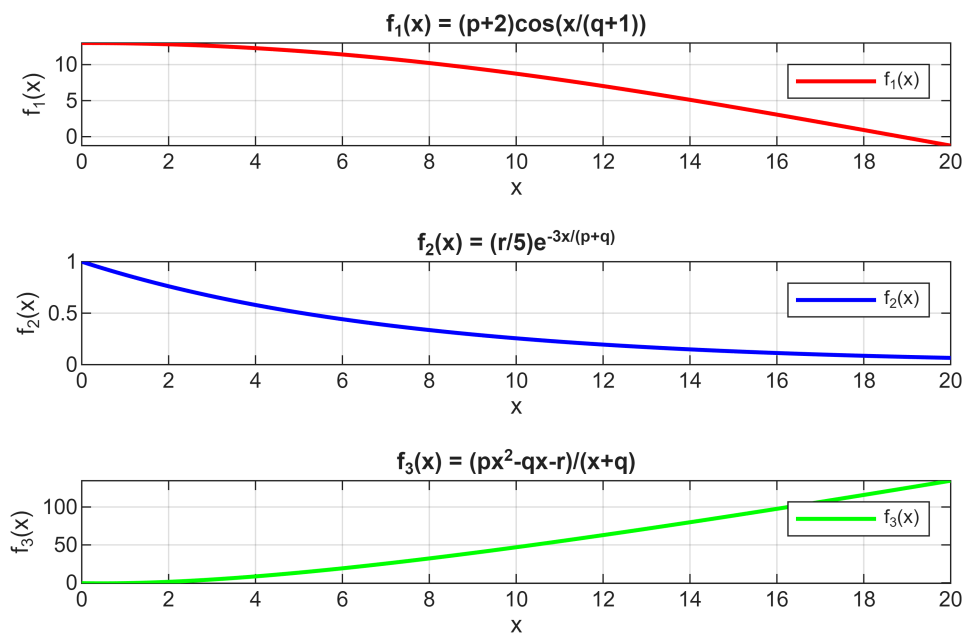
```
f3 = 1×201
    10^2 x
    -0.004545454545455    -0.005396396396396    -0.006035714285714    -0.006469026548673 ...
```

```
figure
tiledlayout(3,1)
nexttile
plot(x, f1, 'r-', 'LineWidth', 1.5)
title('f_1(x) = (p+2)cos(x/(q+1))')
```

```

xlabel('x')
ylabel('f_1(x)')
grid on
legend('f_1(x)')
nexttile
plot(x, f2, 'b-', 'LineWidth', 1.5)
title('f_2(x) = (r/5)e^{-3x/(p+q)}')
xlabel('x')
ylabel('f_2(x)')
grid on
legend('f_2(x)')
nexttile
plot(x, f3, 'g-', 'LineWidth', 1.5)
title('f_3(x) = (px^2-qx-r)/(x+q)')
xlabel('x')
ylabel('f_3(x)')
grid on
legend('f_3(x)')

```



3.2

La señal $s(t) = \sin(2\pi t) + 0.35 \sin(8\pi t)$ es la suma de dos senos de distinta frecuencia. Para identificar esas frecuencias, comparamos cada término contra la forma general $\sin(2\pi ft)$, donde f es la frecuencia en Hz. El primer término, $\sin(2\pi \cdot 1 \cdot t)$, tiene $f=1$ Hz (es decir que completa exactamente un ciclo en 1 segundo, coincidiendo con todo el intervalo evaluado (0 a 1s). El segundo término, $\sin(8\pi t) = \sin(2\pi \cdot 4 \cdot t)$, tiene $f=4$ Hz, entonces ese componente oscila 4 veces más rápido que el primero, completando un ciclo cada 0.25s. Como su amplitud (0.35) es menor que la del primer término (1), este componente actúa como una ondulación rápida que se superpone sobre la forma principal más lenta y de mayor amplitud (que es lo que se observa en la curva de referencia una forma de onda suave de fondo, con pequeñas oscilaciones más rápidas superpuestas).

Ahora con $\Delta t = \frac{0.25}{0.005} = 50$ s se obtienen 50 muestras por cada periodo del componente rápido suficientes para reconstruir fielmente tanto la envolvente principal como el detalle de alta frecuencia el stem prácticamente coincide con la curva continua.

Con $\Delta t = 0.03$ s, el número de muestras por periodo rápido cae a $\frac{0.25}{0.03} \approx 8.3$ que aún alcanza muy bien la forma general, (tiene sentido porque 8.3 sigue estando muy por encima del mínimo que exige el criterio de Nyquist (que dice que para poder reconstruir una señal periódica sin perder información de su forma, necesitas muestrearla más de 2 veces por cada ciclo de su componente de mayor frecuencia $\frac{0.25}{\Delta t} > 2$ es decir que mientras que $\Delta t > 0.125$ s en teoría se conserva la información básica de la señal).

Finalmente con $\Delta t = 0.1$ s $\frac{0.25}{0.1} = 2.5$ las muestras se reducen considerablemente, con un margen estrecho, sigue en el límite de Nyquist pero es de esperarse que el muestreo distorsione la señal y se pierda mucha información en el stem. Concluimos entonces que al aumentar Δt , lo que se pierde es la capacidad de ver los cambios rápidos de la señal (la mayor frecuencia), cuantas menos muestras caben dentro de un periodo de oscilación, mayor es el riesgo de que el muestreo salte por encima de un máximo o mínimo real.

Por último, la diferencia entre plot y stem es que: 'plot' asume conocimiento continuo de la señal, entonces conecta los puntos evaluados con una línea recta entre uno y otro, asumiendo que se conoce el valor de la señal en TODO instante intermedio (o sea que representa el modelo ideal de una señal continua. Mientras que 'stem' refleja únicamente los valores en los instantes discretos de muestreo, sin ninguna interpolación entre ellos (es decir que solo puede conocer la señal en instantes puntuales, nunca de forma continua). Por esta razón es muy útil compararlos en un mismo panel para evaluar visualmente cuándo el muestreo discreto deja de ser una representación confiable de la señal continua de referencia.

```
f0 = d(9)
```

```
f0 =  
1
```

```
t1 = 0:0.005:1
```

```
t1 = 1x201  
0 0.0050000000000000 0.0100000000000000 0.0150000000000000 ...
```

```
s1 = sin(2*pi*f0*t1) + 0.35*sin(8*pi*f0*t1)
```

```
s1 = 1x201  
0 0.075277390825635 0.149831980037013 0.222951906758152 ...
```

```
t2 = 0:0.03:1
```

```
t2 = 1x34  
0 0.0300000000000000 0.0600000000000000 0.0900000000000000 ...
```

```
s2 = sin(2*pi*f0*t2) + 0.35*sin(8*pi*f0*t2)
```

```
s2 = 1x34  
0 0.426972801660766 0.717433907634573 0.805506429950523 ...
```

```
t3 = 0:0.10:1
```

```
t3 = 1x11  
0 0.1000000000000000 0.2000000000000000 0.3000000000000000 ...
```

```
s3 = sin(2*pi*f0*t3) + 0.35*sin(8*pi*f0*t3)
```

```
s3 = 1x11  
0 0.793510090594839 0.618186735591850 1.283926296998457 ...
```

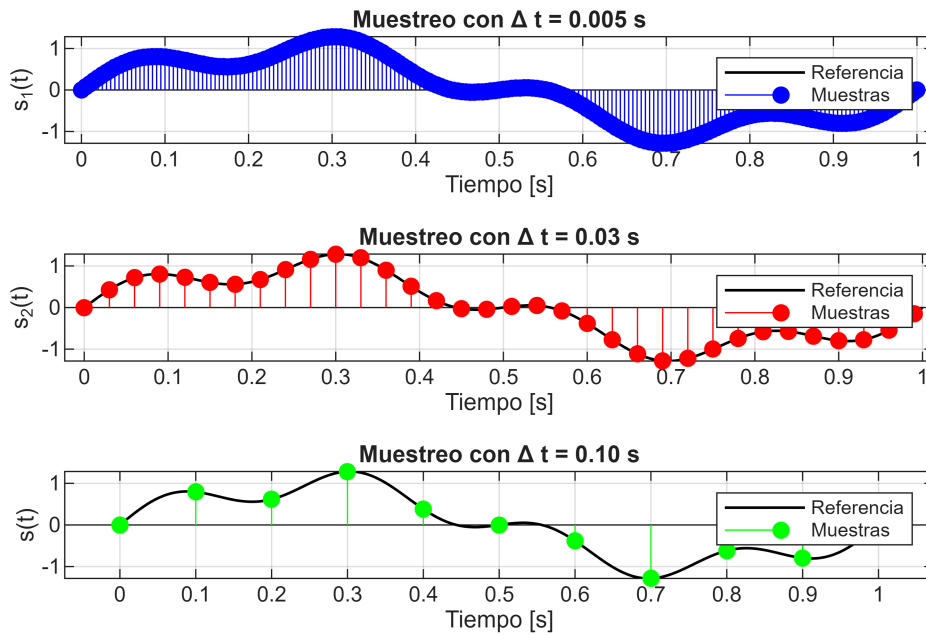
```
referencia_t = 0:0.001:1
```

```
referencia_t = 1×1001
0 0.0010000000000000 0.0020000000000000 0.0030000000000000 ...
```

```
referencia_s = sin(2*pi*f0*referencia_t) + 0.35*sin(8*pi*f0*referencia_t)
```

```
referencia_s = 1×1001
0 0.015078677370727 0.030151551246272 0.045212821650185 ...
```

```
figure
tiledlayout(3,1)
nexttile
plot(referencia_t, referencia_s, 'k-', 'LineWidth', 1)
hold on
stem(t1,s1,'b','filled')
hold off
title('Muestreo con \Delta t = 0.005 s')
xlabel('Tiempo [s]')
ylabel('s_1(t)')
grid on
legend('Referencia', 'Muestras')
nexttile
plot(referencia_t, referencia_s, 'k-', 'LineWidth', 1)
hold on
stem(t2,s2,'r','filled')
hold off
title('Muestreo con \Delta t = 0.03 s')
xlabel('Tiempo [s]')
ylabel('s_2(t)')
grid on
legend('Referencia', 'Muestras')
nexttile
plot(referencia_t, referencia_s, 'k-', 'LineWidth', 1)
hold on
stem(t3, s3,'g', 'filled')
hold off
title('Muestreo con \Delta t = 0.10 s')
xlabel('Tiempo [s]')
ylabel('s(t)')
grid on
legend('Referencia', 'Muestras')
```



Punto 4

4.1

```
M = [ -7   9   35  -3   16   12; 15   5   15   98   54  -6; 18  -1   13   57  -2   55; 22   0
41   33   87   81; 36   7   -4   15   69  -34; -9   65   70  -8   5   25 ];
```

a.

```
B = M(3,6)
```

```
B =
55
```

```
C = M(:,5)
```

```
C = 6x1
16
54
-2
87
69
5
```

```
D = M(2:5,3:6)
```

```
D = 4x4
15   98   54   -6
13   57   -2   55
41   33   87   81
-4   15   69  -34
```

Indexación:

La utilización de la indexación por filas/columnas o indexación lineal son equivalentes en términos prácticos, pues ambas cumplen con la misma utilidad de acceder a trozos de la información de un vector o matriz, sin embargo, es importante contemplar que utilizan diferentes referencias para lo que podrían ser los mismos puntos de información de un arreglo. Mientras que la indexación por filas/columnas utiliza, como indica su nombre, las ubicaciones espaciales de una matriz que es naturalmente bidimensional, es decir, si tenemos un arreglo $A \in \mathbb{R}^{2 \times 2}$, podemos acceder a sus componentes $A_{i,j}$ utilizando la indexación $A(i,j)$ donde i hace referencia a las filas y j a las columnas y $i, j \in \{1, 2\}$. En cuanto a la indexación lineal, la matrix se indexa como si fuera un vector $a \in \mathbb{R}$ que, informalmente, anexa cada fila al ultimo elemento de la anterior para formar una suerte de vector unidimensional a la hora de acceder a sus elementos. Nuevamente, si tenemos una matrix $A \in \mathbb{R}^{2 \times 2}$, con indexación lineal podemos pensarla como $a \in \mathbb{R}$ y acceder al elemento $A_{i,j}$ donde $i, j \in \{1, 2\}$ se traduce a A_k utilizando la indexación $A(k)$ donde $k = i + (j - 1)m$, siendo m es el total de filas cumpliendo $A(i, j) = A(i + (j - 1)m)$.

b.

```
E = diag(M)
```

```
E = 6x1
    -7
     5
    13
    33
    69
    25
```

```
F = M(triu(true(size(M)), 1))
```

```
F = 15x1
     9
    35
    15
    -3
    98
    57
    16
    54
    -2
    87
    12
    -6
    55
    81
   -34
     ⋮
```

Tamaño de F:

Contemplado que $M \in \mathbb{R}^{6 \times 6}$ podemos hacer una prueba a fuerza bruta de la cantidad de elementos en **F**. Retirar la diagonal principal y la sección triangular inferior de la misma, supone eliminar una cantidad $\zeta = \sum_{x=1}^i x$ de la matriz, donde $i = 6$ es la cantidad de filas/columnas, pues de cada fila se retira una cantidad de elementos equivalente al número de fila, la sumatoria resulta en $\zeta = 21$ por lo que $|F| = 6 \cdot 6 - \zeta = 15$, equivalente a la cantidad de elementos en **F**.

c.

```
negativos = M(M<0);
[filas_neg, columnas_neg] = find(M < 0);
disp('Valores negativos de M'); disp(negativos')
```

Valores negativos de M

-7	-9	-1	-4	-3	-8	-2	-6	-34
----	----	----	----	----	----	----	----	-----

```
disp('Posiciones de negativos:'); disp([filas_neg columnas_neg])
```

Posiciones de negativos:

1	1
6	1
3	2
5	3
1	4
6	4
3	5
2	6
5	6

```
media = mean(M(:));
fprintf('Media de M: %.4f\n', media);
```

Media de M: 24.5556

```
mayores_media = M(M > media);
[filas_mayor_media, columnas_mayor_media] = find(M > media);
disp('Valores mayores que la media:'); disp(mayores_media')
```

Valores mayores que la media:

36	65	35	41	70	98	57	33	54	87	69	55	81	25
----	----	----	----	----	----	----	----	----	----	----	----	----	----

```
disp('Posiciones de mayores que la media:'); disp([filas_mayor_media columnas_mayor_media])
```

Posiciones de mayores que la media:

5	1
6	2
1	3
4	3
6	3
2	4
3	4
4	4
2	5
4	5
5	5
3	6
4	6
6	6

```
[valores,indices] = sort(abs(M(:)), 'descend');
top3_valores = M(indices(1:3));
top3_posiciones = indices(1:3);
[top3_fila, top3_columna] = ind2sub(size(M), top3_posiciones);
disp('Top 3 valores de mayor magnitud absoluta:'); disp(top3_valores')
```

Top 3 valores de mayor magnitud absoluta:

98	87	81
----	----	----

```
disp('En posiciones:'); disp([top3_fila top3_columna]);
```

En posiciones:

2	4
4	5
4	6

d.

```
mu = mean(M,1);
sigma = std(M,0,1);
Z = (M - mu)./sigma;
media_Z = mean(Z,1);
sigma_Z = std(Z,0,1);
disp('Z (matriz normalizada) ='); disp(Z)
```

```
Z (matriz normalizada) =
-1.117850267774412 -0.204973742220589 0.255680064184295 -0.869796782408251 -0.601483692061360 -0.2451567306
0.143314136894155 -0.363663091036529 -0.511360128368590 1.640188218255560 0.429631208615257 -0.6792047128
0.315291101167142 -0.601697114260439 -0.588064147623879 0.621283416005894 -1.089906539750283 0.7917356712
0.544593720197791 -0.562024777056454 0.485792121950161 0.024851336640236 1.325073096044951 1.4186938677
1.347152886805061 -0.284318416628559 -1.240048311293831 -0.422472722884008 0.836650248356027 -1.3543904629
-1.232501577289737 2.016677141202571 1.598000401151844 -0.994053465609430 -0.899964321204591 0.0683223675
```

```
disp('Media de cada columna de Z:'); disp(media_Z)
```

Media de cada columna de Z:

1.0e-15 *

-0.037007434154172 0.074014868308344 0.111022302462516 -0.018503717077086 0.055511151231258 -0.0832667268

```
disp('Desv. estándar de cada columna de Z:'); disp(sigma_Z)
```

Desv. estándar de cada columna de Z:

1.000000000000000 1.000000000000000 1.000000000000000 1.000000000000000 1.000000000000000 1.000000000000000

Normalización:

La normalización prueba ser efectiva para alinear cada columna muy cerca de una media 0 y con una desviación estandar de 1 como es esperado de las formulas. Este resultado es supremamente útil al utilizar variables con coeficientes asociados de diferentes ordenes de magnitud, para problemas de inversión u optimización como podría ser mínimos cuadrados o cualquier otra aproximación para la inversa de una matriz, una normalización de este tipo permite eliminar sesgos del problema, pues todas las variables tienen el mismo peso (1) en sus coeficientes.

e.

```
sub1 = [M(3,1) M(4,3); M(1,4) M(5,1)]
```

```
sub1 = 2x2
18 41
-3 36
```

```
sub2 = [M(2,3) M(2,5); M(6,3) M(1,5)]
```

```
sub2 = 2x2
15 54
70 16
```

```
sub3 = [M(1,1) M(6,6); M(2,2) M(3,3)]
```

```
sub3 = 2x2
    -7    25
     5    13
```

```
sub_transpuesta = sub3'
```

```
sub_transpuesta = 2x2
    -7     5
    25    13
```

```
fprintf('Dimensiones: sub1=%dx%d, sub2=%dx%d, sub3'='%dx%d\n',size(sub1,1), size(sub1,2),
size(sub2,1), size(sub2,2), ...
size(sub3',1), size(sub3',2));
```

```
Dimensiones: sub1=2x2, sub2=2x2, sub3'=2x2
```

```
G = sub1 * sub2 + sub3'
```

```
G = 2x2
    3133    1633
    2500     427
```

```
verificacion_G11 = sub1(1,1)*sub2(1,1) + sub1(1,2)*sub2(2,1) + sub3(1,1);
fprintf('Verificación manual G(1,1) = %d | MATLAB: G(1,1) = %d\n', verificacion_G11, G(1,1));
```

```
Verificación manual G(1,1) = 3133 | MATLAB: G(1,1) = 3133
```

4.2

Funciones auxiliares:

El algoritmo se separa en cuatro tareas: localizar letras iniciales candidatas, identificar direcciones posibles, validar los límites de la matriz y verificar letra por letra si una palabra está presente en una dirección determinada.

```
%Busca desde una posición inicial, la siguiente celda de la sopa de letras que pueda ser el comienzo
de al menos una palabra
```

```
function [fila_salida,columna_salida, palabras_a_buscar] = continuar_recorrido(fila_entrada,
columna_entrada, S, palabras)
    salto_linea = false;
    encontrado = false;
    i = fila_entrada;
    % Recorre las filas desde la fila inicial hasta el final de la matriz.
    while i <= length(S) && ~encontrado
        if ~salto_linea
            j = columna_entrada;
        else
            j = 1;
        end
        % Recorre las columnas de la fila actual.
        while j <= length(S(i,:)) && ~encontrado
            letra_actual = S(i,j);
            palabras_a_buscar = {};
            % Identifica cuáles palabras pueden iniciar con letra_actual.
            for indice_palabra = 1:numel(palabras)
                if palabras{indice_palabra}(1) == letra_actual
                    palabras_a_buscar{end+1}=palabras{indice_palabra};
                    encontrado = true;
                    fila_salida = i;
                end
            end
        end
        i = i + 1;
    end
```

```

        columna_salida = j;
    end
end
j=j+1;
end
salto_linea = true;
i=i+1;
end
end

% Comprueba si alguna de las palabras candidatas está presente en la sopa de letras, partiendo de una
% posición y siguiendo las direcciones indicadas.
function palabras_encontradas = verificar_palabra(fila_entrada, columna_entrada, direcciones, S,
palabras, palabras_a_anexar)
    n = 2;
    fallo = false;
    i = 1;
    no_hay_direcciones = false;
    % Punto inicial
    fila_inicio=fila_entrada;
    columna_inicio=columna_entrada;
    % Evalúa cada una de las direcciones candidatas.
    while i <= length(direcciones(:,1)) && ~no_hay_direcciones
        fila_entrada = fila_inicio;
        columna_entrada = columna_inicio;
        n=2;
        fallo = false;
        % Dirección de desplazamiento
        delta_i = direcciones(i, 1);
        delta_j = direcciones(i, 2);
        if ~(delta_i ==0 && delta_j==0)
            indice_palabras = 1;
            % Verifica cada palabra que puede empezar en la celda inicial
            while indice_palabras <= numel(palabras)
                fallo = false;
                fila_entrada = fila_inicio;
                columna_entrada = columna_inicio;
                n=2;
                "EMPIEZA ANALISIS PALABRA: " + palabras{indice_palabras} + ", Dir: (" +
                delta_i+","+delta_j+")";
                % Evita analizar una palabra ya encontrada.
                encontrar = contains(palabras_a_anexar, palabras{indice_palabras}, 'IgnoreCase', true);
                if sum(encontrar) > 0
                    "YA SE ENCONTRO";
                    fallo = true;
                end
                % Compara las letras restantes siguiendo la dirección actual
                while n <= length(palabras{indice_palabras}) && ~fallo
                    if mirar_validez(fila_entrada+delta_i, columna_entrada+delta_j, S)
                        if S(fila_entrada+delta_i, columna_entrada+delta_j) == palabras{indice_palabras}
                            (n)
                                n=n+1;
                                fila_entrada = fila_entrada+delta_i;
                                columna_entrada = columna_entrada+delta_j;
                                palabras{indice_palabras}(n-1) +", " + S(fila_entrada,
                                columna_entrada);
                            else
                                fallo = true;

```



```

        x = fila_entrada+delta_i;
        y=columna_entrada+delta_j;
        "FALLO, Palabra: " + palabras{indice_palabras} + ", Pos: " + x + "," + y;
        palabras{indice_palabras}(n) +", " + S(x, y);
    end
else
    "FALLO EN VALIDEZ";
    fallo = true;
end
end
% Si se verificaron todas las letras, se registra la palabra.
if ~fallo
    "ENCONTRADA";
    palabras{indice_palabras};
    palabras_a_anexar = [palabras_a_anexar, palabras{indice_palabras} + ", Inicio:
" + fila_inicio+", "+columna_inicio+", Final: "+fila_entrada+", "+columna_entrada+", Dir:" +
delta_i+", "+delta_j];
    end
    indice_palabras = indice_palabras+1;
end
else
    no_hay_direcciones = true;
end
i=i+1;
end
palabras_encontradas=palabras_a_anexar;
end
% Verifica si una posición pertenece a los límites de la matriz de la sopa de letras la posición es
válida si sus índices son positivos y no superan las dimensiones de la matriz.
function [valido] = mirar_validez(fila_entrada,columna_entrada, S)
    if fila_entrada>0 && fila_entrada<length(S) && columna_entrada>0 &&
columna_entrada<length(S(fila_entrada,:))
        valido=true;
    else
        valido=false;
    end
end
end
% Identifica las direcciones desde una posición inicial hacia las celdas vecinas que contienen la
segunda letra de alguna posible palabra.
function posibles_direcciones = mirar_vecinos(fila_entrada,columna_entrada, palabras, radio, S)
    posibles_direcciones = zeros(0,2);
    %Recorrido de filas y columnas
    i = fila_entrada-radio;
    while i <= fila_entrada+radio
        j = columna_entrada-radio;
        while j <= columna_entrada+radio
            if mirar_validez(i,j, S) && (i ~= fila_entrada || j ~= columna_entrada)
                % Compara la letra vecina con la segunda letra de cada palabra
                for indice_palabras=1:numel(palabras)
                    %Si coincide se almacena el desplazamiento
                    if S(i,j) == palabras{indice_palabras}(2)
                        posibles_direcciones(end+1,:)= [i-fila_entrada, j-columna_entrada];
                    end
                end
                j=j+1;
            else
                j = j+1;
            end
        end
    end
end

```

```

        end
        i=i+1;
    end
end
end

```

Solución

```

load('sopa_de_letras.mat');
palabras

```

```

palabras = 1x7 cell
'INDEXACION' 'MATRIZ'      'MATLAB'      'COMPUTACION' 'CIENTIFICA' 'ECUACION'  'P...'

```

```

palabras_encontradas={};
i = 1;
fila_inicial=1;
columna_inicial=1;
finalizo = false;
while length(palabras_encontradas) ~= length(palabras) && ~finalizo
    [fila_inicial, columna_inicial, palabras_a_mirar] = continuar_recorrido(fila_inicial,
columna_inicial, S, palabras);
    posibles_direcciones = mirar_vecinos(fila_inicial, columna_inicial, palabras_a_mirar, 1, S);
    palabras_encontradas=verificar_palabra(fila_inicial, columna_inicial, posibles_direcciones, S,
palabras_a_mirar, palabras_encontradas);
    if columna_inicial < length(S(1,:))
        columna_inicial=columna_inicial+1;
    else
        if fila_inicial < length(S(:,1))
            columna_inicial=1;
            fila_inicial = fila_inicial+1;
        else
            finalizo = true;
        end
    end
end
palabras_encontradas

```

```

palabras_encontradas = 1x7 string
'INDEXACION, Inicio: 2,3, Final: 2,12, Dir...' 'CIENTIFICA, Inicio: 3,14, Final: 1...'

```

```

function visualizeSoup(sopa, data)
[rows, cols] = size(sopa);
figure('Position',[100 100 800 500])
hold on
% Grid
for x = 0.5:1:cols+0.5
    plot([x x], [0.5 rows+0.5], 'w');
end
for y = 0.5:1:rows+0.5
    plot([0.5 cols+0.5], [y y], 'w');
end
% Pintar las celdas
for k = 1:length(data)
    tokens = regexp(data{k}, ...
        '(\w+), Inicio: (-?\d+),(-?\d+), Final: (-?\d+),(-?\d+), Dir: (-?\d+),(-?\d+)', ...
        'tokens');
    tokens = tokens{1};

```

```

word = tokens{1};
start = [str2double(tokens{2}), str2double(tokens{3})];
final = [str2double(tokens{4}), str2double(tokens{5})];
dir = [str2double(tokens{6}), str2double(tokens{7})];
fprintf('%s: [%d,%d] -> [%d,%d]\n', ...
    word, start, final);
pos = start;
while true
    row = pos(1);
    col = pos(2);
    rectangle( ...
        'Position', [col-0.5, row-0.5, 1, 1], ...
        'FaceColor', 'yellow', ...
        'EdgeColor', 'none');
    if isequal(pos, final)
        break
    end
    pos = pos + dir;
end
end

% Letras
for i = 1:rows
    for j = 1:cols
        text(j, i, sopa(i,j), ...
            'HorizontalAlignment', 'center', ...
            'VerticalAlignment', 'middle', ...
            'FontSize', 11, ...
            'FontWeight', 'bold');
    end
end

axis equal
xlim([0.5 cols+0.5])
ylim([0.5 rows+0.5])
set(gca, 'YDir', 'reverse')
set(gca, 'XTick', [])
set(gca, 'YTick', [])
hold off
end
visualizeSoup(S,palabras_encontradas);

```

```

INDEXACION: [2,3] -> [2,12]
CIENTIFICA: [3,14] -> [12,5]
MATRIZ: [5,20] -> [10,20]
MATLAB: [14,3] -> [19,8]
ECUACION: [18,4] -> [11,11]
COMPUTACION: [22,22] -> [22,12]
PUNTOFLOTANTE: [24,1] -> [12,1]

```

D	L	Z	O	O	Y	O	X	I	S	U	K	L	J	I	A	X	Z	B	D	U	H	C	F
W	C	I	N	D	E	X	A	C	I	O	N	A	T	I	G	S	W	G	E	J	P	A	B
T	T	Q	Q	F	J	C	Y	X	Q	T	O	G	C	B	P	I	H	C	X	X	I	V	S
V	I	F	G	K	Q	K	Y	T	N	T	T	I	G	A	A	F	I	R	V	S	N	Z	Y
E	T	Y	X	N	F	J	C	E	V	R	E	N	I	W	K	S	F	X	M	D	M	X	O
D	C	P	H	I	U	V	A	Q	P	N	E	I	F	M	R	C	Y	D	A	E	I	S	K
I	C	U	I	F	Q	Z	N	G	T	I	L	U	K	I	D	H	L	T	T	O	O	Q	T
Z	Y	R	H	D	J	X	K	I	W	X	J	H	M	B	Q	C	S	P	R	E	P	W	U
P	U	E	K	O	B	Z	F	U	L	I	Z	J	Z	D	I	Y	G	M	I	R	K	R	S
G	H	D	L	S	L	I	N	P	N	W	H	Z	Q	D	Z	N	K	S	Z	L	W	V	B
Y	O	A	O	S	C	B	O	D	J	N	K	V	N	F	N	H	R	P	V	J	K	D	T
E	J	K	T	A	I	N	S	D	O	D	Q	B	K	G	Y	W	L	W	C	N	J	L	E
T	K	I	T	L	Z	Q	X	I	M	V	Y	P	W	Q	B	J	L	O	Z	X	T	B	X
N	R	M	S	L	J	D	C	I	N	C	S	O	L	S	K	S	I	S	H	B	Z	I	E
A	T	Y	A	V	H	A	Q	S	A	L	J	F	Z	Z	Q	R	X	X	Q	Z	Z	V	O
T	R	A	S	T	U	M	L	P	N	E	J	H	W	O	S	B	T	K	Y	V	O	M	P
O	T	F	G	C	L	G	S	J	Y	C	M	R	Q	Y	E	H	I	Z	R	D	Y	K	P
L	B	Y	E	E	L	A	L	O	N	P	M	R	X	K	S	G	P	P	F	X	I	J	S
F	C	A	K	Y	L	F	B	P	Z	Z	D	I	Q	O	E	Q	G	I	P	D	J	X	U
O	A	W	O	A	X	F	D	D	V	P	Q	B	W	K	S	T	G	B	S	U	Q	P	U
T	K	Y	X	N	C	Z	T	R	O	W	D	A	S	H	T	Y	N	D	F	K	Y	K	U
N	B	H	V	F	Y	B	B	P	L	P	N	O	I	C	A	T	U	P	M	O	C	S	Y
U	I	J	F	B	K	Y	D	M	Z	S	V	Y	M	L	Z	P	X	O	Z	L	E	C	D
P	U	E	F	H	G	Z	O	X	U	Z	G	P	K	H	T	X	I	S	W	X	L	U	M

Análisis:

Verificación de límites

La función auxiliar *mirar_validez* es la encargada de revisar los límites de la matriz. Recibe una fila y columna candidatas y devuelve *true* únicamente si ambas están en el rango válido de la matriz, es decir, dentro de las dimensiones de la misma. Esto garantiza que al intentar evaluar, por ejemplo, $S(0, j)$ o $S(i, \text{columnas}+1)$, no se produzca un error de indexación.

Búsquedas horizontales, verticales y diagonales

El tipo de búsqueda se define en la función auxiliar *mirar_vecinos* que depende de los desplazamientos Δi y Δj , obtenidos de la matriz *direcciones*. Los desplazamientos se generan dinámicamente comparando la posición inicial con los vecinos que son letras que coinciden la siguiente de alguna palabra no encontrada.

- En una búsqueda **horizontal**, la fila no cambia $\Delta i = 0$ y solo cambia la columna. $(0, 1)$ busca hacia la derecha, mientras que $(0, -1)$ busca hacia la izquierda.
- En una búsqueda **vertical**, la columna no cambia $\Delta j = 0$ y solo cambia la fila. $(1, 0)$ busca hacia arriba y $(-1, 0)$ hacia abajo.
- En una búsqueda **diagonal**, cambian simultáneamente la fila y la columna. $(1, 1)$ es una búsqueda en la diagonal superior derecha y $(-1, -1)$, busca en una diagonal inferior izquierda.

Letras repetidas y candidatos

Dado que el algoritmo no logra discernir realmente cual aparición de una letra pertenece a una palabra buscada, las letras repetidas producen varios candidatos. La función auxiliar *continuar_recorrido* recorre la sopa, al encontrar una letra que coincide con la primera letra de al menos una de las palabras buscadas, guarda todas las palabras formadas desde esa letra.

La función auxiliar *mirar_vecinos*, como indica su nombre, revisa las posiciones contiguas, si los vecinos contienen la segunda letra de una posible palabra, se genera una dirección de búsqueda.

Finalmente, la función auxiliar *verificar_palabra* evalúa las combinaciones encontradas y las direcciones letra a letra. Si las letras coinciden consecutivamente con una de las palabras buscadas, se acepta la candidata, de lo contrario se descarta.

Conclusiones

La diferencia entre la unidad angular resultó inmensa: E1 en grados dio 0.6653 frente a 0.0551 en radianes, bajo el mismo valor de $\theta=42$, una diferencia de más de un orden de magnitud, debida exclusivamente a la unidad angular asumida por la función trigonométrica.

En el segundo apartado se confirmó la teoría de álgebra matricial conocida donde $AB \neq BA$ obteniendo $AB(2,1)=420$ frente a $BA(2,1)=316$. Así como comprobamos la incompatibilidad dimensional $A(3 \times 3) \times C(2 \times 3)$ incompatible si en $A(m \times n)$ y $C(q \times p)$ $n \neq q$, en este punto se resolvió tal incompatibilidad con dos correcciones ($C \times A$ de 2×3 y $A \times C'$ de 3×2), verificadas antes y después de cada corrección; así como se conoció la importancia de la función 'try-catch' útil para que un error no detenga toda la ejecución del código y dar espacio a posibles correcciones de dicho error que resalta MATLAB.

En el punto 3 las funciones f_1 , f_2 , f_3 mostraron comportamientos distintos en el intervalo de $x = [0, 20]$: f_1 mostró una oscilación acotada $\in [-13, 13]$, y un cruce con el eje horizontal en $x \approx 18.85$, f_2 mostró un decaimiento exponencial (coherente con su exponente negativo de 1 a 0.065, con $\tau = 7.33$, y f_3 un crecimiento cuasi-lineal de -0.45 a 134.68 sin discontinuidades en el dominio evaluado. En cuanto al muestreo de $s(t)$ combinó dos componentes de frecuencia distinta (1 Hz y 4 Hz) en la cual el muestreo se evaluó respecto al componente de frecuencia más exigente (4 Hz, con periodo de 0.25s), los tres Δt evaluados cumplieron formalmente el criterio de Nyquist (8.3, 50 y 2.5 muestras/ciclo respectivamente), pero el stem de $\Delta t = 0.10s$, mostró una distorsión visible frente a la señal de referencia y a las otras dos muestras evaluadas previamente.

En el Punto 4 se aplicaron diferentes técnicas de indexación y manipulación de matrices. La indexación por filas y columnas permitió extraer elementos y submatrices mediante comandos del estilo $M(i,j)$, mientras que la indexación lineal permitió acceder a elementos con un único índice $M(k)$. MATLAB realiza esta última indexación por columnas. La indexación lógica permitió identificar 9 valores negativos y 14 valores mayores que la media global de 24.5556. Además, la función *find* permitió obtener sus posiciones, mientras que *ind2sub* facilitó convertir índices lineales en coordenadas de fila y columna. La normalización por columnas produjo medias cercanas a cero y desviaciones estándar cercanas a uno. Por último, la sopa de letras aplicó estos conceptos en un problema de búsqueda. El algoritmo valida los límites de la matriz, evalúa direcciones horizontales, verticales y diagonales, y descarta candidatos cuando las letras no coinciden con la palabra completa.

Referencias

[1] The MathWorks, Inc., "Create Live Scripts in the Live Editor", documentación oficial de MATLAB.

[2] The MathWorks, Inc., "Array Indexing", documentación oficial de MATLAB.

[3] The MathWorks, Inc., documentación de plot, stem, tiledlayout, fill, load y reshape